Chapter Four
Test Planning

Test plan

- A test plan is a document that outlines the strategy, objectives, resources, and schedule of a software testing process.
  - The test plan will typically include details such as **the type and number of tests that need to be conducted, the purpose of each test, the required tools, and how test results will be analyzed and reported.**
- It is regularly updated throughout the testing process to reflect any discoveries or changes in strategy.

Why are Test Plans important?

- A test plan is essential for several reasons.
  - Firstly, it is a communication tool between stakeholders and testing team members.
  - This ensures that everyone understands **what, why, and how** to test.
  - It will also outline how to report test findings, what to consider as a pass or fail, and any other criteria that may be applicable.
  - It will outline the expected outcomes and ensure that testing happens according to plan.

The 7 Steps to Create a Test Plan (Based on IEEE 829)

- According to the IEEE 829 standard, the process of creating a test plan involves seven key steps
  - Analyze the Product Structure and Architecture
  - Design the Test Strategy
  - Define All Test Objectives
  - Define the Test Areas (Scope)
  - Define All Available Resources
  - Schedule All Test Activities
  - Determine All Test Deliverables

1. Analyze the Product Structure and Architecture

- **What to do:** Understand the software's internal design and how its components interact. This helps you identify what needs to be tested at a granular level.
- **Key Question:** What are the main modules, functions, and technical dependencies of this application?

2. Design the Test Strategy

- **What to do:** Decide on the overall approach to testing. This includes choosing the testing levels (e.g., unit, integration, system), techniques (e.g., black-box, white-box), and types of tests (e.g., functional, performance, security) you will use.
- **Key Question:** How will we test the product? Will we use automated or manual testing for different areas?

3. Define All Test Objectives

- **What to do:** Clearly state the high-level goals of the testing effort. What are you trying to achieve or prove about the software?
- **Key Question:** Is our goal to verify all functional requirements, assess system performance under load, or validate the user experience?

4. Define the Test Areas (Scope)

- **What to do:** Specify what will be tested and, just as importantly, what will *not* be tested. This sets clear boundaries for the team.
- **Key Question:** Which features are in scope for this test cycle? Which are out of scope (e.g., legacy features, third-party integrations)?

5. Define All Available Resources

- **What to do:** Identify all the people, tools, environments, and equipment needed to execute the test plan. This is a critical step for accurate scheduling.
- **Key Question:** Who are the testers? What test management tools, hardware, and software environments do we have access to?

6. Schedule All Test Activities

- **What to do:** Create a timeline for all testing tasks. A good practice is to use relative dates or entry/exit criteria instead of fixed dates to make the schedule more resilient to changes.
- **Key Question:** What are the milestones? When will test design start, when will test execution begin, and what conditions must be met to move from one phase to the next?

7. Determine All Test Deliverables

- **What to do:** Define the outputs of the testing process. This is not just about finding bugs but also about documenting what was done and what the results were.
- **Key Question:** What documents will be produced? (e.g., test plans, test cases, test reports, defect logs).

Test plan documentation

- To explain the documentation process, we will be using an **E-commerce Platform** called **ShopEase**.
  - It covers both **functional** and **non-functional** testing, and each section includes a detailed explanation so you can use it when you right your test plan for your respective project.

- Front page explains the
  - Project, version, date
- **Example**
  - **Project:** ShopEase v2.0
  - **Version:** 1.0
  - **Date:** 2025-09-01

1. Test Plan Identifier

- A unique identifier for this test plan, including version and project name, ensures traceability across documentation.
  - **Forward Traceability (from Requirements to Testing)**
    - The test plan identifier is referenced in **test case specifications**, so each test case knows which test plan it belongs to.
  - **Backward Traceability (from Testing to Requirements)**
    - When a defect is logged, it often includes the test plan ID. This allows you to trace back which test plan was used, which test case failed, and ultimately which requirement was affected.
- **Example** : **ID:** TP-SHOPEASE-2.0-001

2. Introduction & Scope

- The scope defines the boundaries of testing. It clarifies what is included and what is explicitly excluded to manage expectations.
- **Example:**
  - **Scope:**
    - This Test Plan covers all testing activities for ShopEase v2.0, a full-featured e-commerce platform. It includes functional testing of new features (e.g., product search, cart, checkout) as well as non-functional testing (performance, security, usability, compatibility). The plan addresses testing at the system and acceptance levels.
  - **Out of Scope:**
    - Unit testing (handled by developers), third-party payment gateway certification (done by vendor), and hardware compatibility testing beyond standard desktop/mobile browsers.

3. Test Items

- Listing the test items (software components, interfaces, and infrastructure) ensures the team understands the full scope of what will be exercised.
- **Example**
  - **Frontend:** Web application (React), mobile responsive views
  - **Backend:** REST APIs (Node.js), microservices (catalog, cart, order, user)
  - **Database:** PostgreSQL schemas and data migrations
  - **Integrations:** Payment gateway (Stripe), email service (SendGrid), SMS notifications
  - **Infrastructure:** Cloud environment (AWS), CI/CD pipeline artifacts

4. Features to Be Tested (Functional)

- Functional features are the core business capabilities.
- They are derived from requirements/user stories and will be validated through test cases.

- Example:

Feature Area	Description
User Management	Registration, login, profile update, password reset, address management
Product Catalog	Browse, search, filters, product details, reviews, ratings
Shopping Cart	Add/remove items, update quantities, save for later, price calculation
Checkout	Shipping method selection, payment, order confirmation, email receipts
Order Management	Order history, order details, cancellation, returns
Admin Panel	Product management, inventory updates, order processing, user management

5. Non-Functional Features to Be Tested

- Non-functional requirements are often just as critical as functional ones.
- This section makes them explicit so they are not overlooked.

Type	Description
Performance	Response times under load, concurrent user handling, API throughput
Scalability	System’s ability to scale horizontally with increased traffic
Security	Authentication/authorization, data encryption, OWASP Top 10 vulnerabilities (SQL injection, XSS, etc.)
Usability	Intuitive navigation, accessibility (WCAG 2.1 AA), consistency across devices
Compatibility	Browsers (Chrome, Firefox, Safari, Edge), devices (desktop, tablet, mobile), operating systems
Reliability	Error handling, graceful degradation, recovery from failures
Maintainability	Code quality, logging, monitoring readiness (not tested by QA, but verified)

6. Test Approach / Strategy

- The strategy defines how testing will be executed, what tools will be used, and how defects will be managed.
- It aligns the team on the overall testing methodology.

- **Example**
  - **Testing Levels:**
    - **Integration Testing:** Verify interactions between microservices and third-party APIs.
    - **System Testing:** End-to-end functional testing of all features in a staging environment.
    - **Acceptance Testing:** Business-user validation against acceptance criteria.
  - **Test Types:**
    - **Functional:** Black-box testing using equivalence partitioning, boundary value analysis, decision tables, and state transition.
    - **Performance:** Load testing (JMeter) to simulate 1000 concurrent users, stress testing to find breaking point.
    - **Security:** Static code analysis (SAST), dynamic scanning (DAST), manual penetration testing.
    - **Usability:** Heuristic evaluation and moderated user sessions.
    - **Regression:** Automated regression suite (Selenium/Cypress) run on each build.

- **Tools:**
  - Test management: TestRail
  - Defect tracking: Jira
  - Automation: Selenium WebDriver, Cypress, Postman (API), JMeter
  - Security: OWASP ZAP, SonarQube
- **Defect Management:**
  - All defects logged with severity (Blocker, Critical, Major, Minor) and priority; tracked in Jira with daily triage.

7. Entry Criteria

- Entry criteria prevent starting testing prematurely, which can waste resources and yield unreliable results.
- **Example : Before test execution begins**
  - All functional and non-functional requirements are finalized and approved.
  - Test environment (staging) is deployed with build v2.0.
  - Test data (users, products, orders) is seeded.
  - Test cases for all functional areas have been reviewed and approved.
  - Performance test scripts are validated with baseline metrics.

8. Exit Criteria

- Exit criteria provide measurable, objective conditions to declare testing finished, preventing endless cycles of retesting.
- **Example: Testing is considered complete when**
  - All planned test cases have been executed (functional and non-functional).
  - No open Blocker or Critical defects remain.
  - Major defects are resolved or deferred with stakeholder approval.
  - Performance targets (e.g., 95th percentile response time < 2s) are met.
  - Security scan shows no high-risk vulnerabilities.
  - Usability issues with severity > medium have been addressed.
  - Test summary report is delivered and signed off.

9. Resources & Responsibilities

- Clearly defined roles and responsibilities ensure accountability and smooth collaboration.
- Tools and test environments need should be mentioned.
- **Example**

Role	Name / Team	Responsibilities
Test Manager	Jane Smith	Overall test planning, coordination, reporting
Functional Test Lead	John Doe	Manage functional test execution, test case reviews
Performance Test Lead	Emily Wang	Design and execute load/stress tests, report metrics
Security Test Lead	Alex Chen	Coordinate security testing, vulnerability assessment
Automation Engineer	Lisa Park	Develop and maintain automated regression suites
Test Analysts	Team of 5	Write and execute test cases, report defects
DevOps	IT Ops	Maintain test environment, support CI/CD
Product Owner	Mark Lee	Provide clarifications, accept exit criteria

10. Schedule

- A realistic schedule with overlapping phases accounts for dependencies and allows parallel work (e.g., functional testing may continue while performance testing starts).
- **Example:**

Phase	Start Date	End Date
Test planning	Apr 1	Apr 5
Test case design (functional)	Apr 6	Apr 12
Test case design (non-functional)	Apr 6	Apr 12
Environment setup & test data	Apr 8	Apr 12
Functional test execution	Apr 13	Apr 20
Performance test execution	Apr 18	Apr 22
Security test execution	Apr 18	Apr 24
Usability testing	Apr 21	Apr 23
Defect retesting & regression	Apr 21	Apr 27
Acceptance testing	Apr 28	Apr 30
Test summary report	May 1	May 2

11. Risks & Contingencies

- Proactive risk identification and mitigation plans reduce surprises and allow the team to respond quickly.
- **Example**

Risk	Impact	Probability	Mitigation
Test environment instability	High	Medium	Use dedicated staging environment; coordinate with DevOps for 24/7 support; have fallback environment.
Delayed builds from development	High	Medium	Implement daily integration builds; run smoke tests early to catch showstoppers.
Performance test environment not matching production	Medium	High	Right-size environment; use production data anonymization; document differences and adjust targets.
Security testing reveals major vulnerabilities late	High	Low	Shift left: run SAST during development; include security test cases earlier in cycle.
Resource availability (holidays)	Medium	Medium	Cross-train team members; plan schedule around known leaves.

12. Test Deliverables

- Deliverables are the tangible outputs of the testing process. Listing them ensures nothing is forgotten and sets expectations with stakeholders.
- **Example:**
  - Master Test Plan (this document)
  - Detailed Test Cases (TestRail)
  - Automated Test Scripts (GitHub repository)
  - Performance Test Report (including metrics and recommendations)
  - Security Test Report (vulnerability scan results, penetration test findings)
  - Usability Evaluation Report
  - Defect Log (Jira)
  - Test Execution Summary (daily/weekly status)
  - Test Summary Report (final sign-off)….

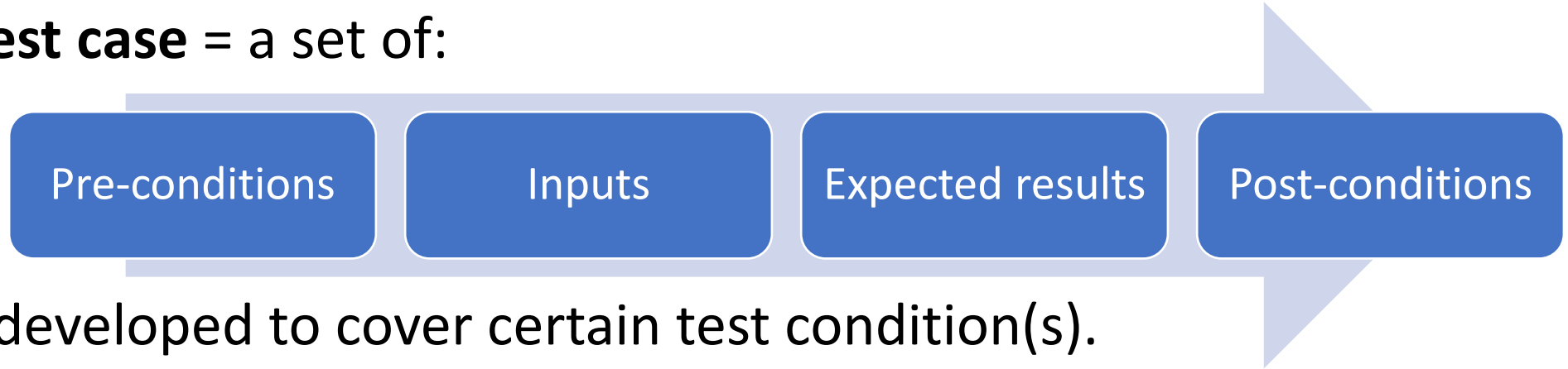
2. Test design

- During test design:



are created and specified.

- **Test case** = a set of:



developed to cover certain test condition(s).

Test cases

- A **test case**, is a set of conditions under which a tester will determine whether an application, software system or one of its features is working as it was originally established for it to do.
- Test cases are often referred to as test scripts, particularly when written – when they are usually collected into test suites.
- A Test Case is a set of actions executed to verify a particular feature or functionality of your software application.

Test cases

- A test case is a description of a specific interaction that a tester will have in order to test a single behavior of the software.
- Test cases are very similar to use cases, in that they are step-by-step narratives which define a specific interaction between the user and the software.
- A typical test case is laid out in a table, and includes:
  - A unique name and number
  - A requirement which this test case is exercising
  - Preconditions which describe the state of the software before the test case
  - Steps that describe the specific steps which make up the interaction
  - Expected Results which describe the expected state of the software after the test case is executed

Test cases

- Test cases must be repeatable.
- Good test cases are data-specific, and describe each interaction necessary to repeat the test exactly.

- Example
- Test Case Design (for Login Feature):
- Positive test case

Field	Value
TC-ID	LOGIN_001
Title	Login with valid username and password
Preconditions	User is registered (e.g., testuser@example.com / Pass123!). Login page is displayed.
Test Steps	1. Enter valid email: testuser@example.com 2. Enter valid password: Pass123! 3. Click "Login" button.
Expected Result	User is redirected to the dashboard. Welcome message shows username. No error messages.
Actual Result	(To be filled after execution)
Status	Pass / Fail

- Negative test case

Field	Value
TC-ID	LOGIN_002
Title	Login with invalid email format (missing "@")
Preconditions	Login page is displayed.
Test Steps	1. Enter testuser.example.com 2. Enter any password 3. Click "Login".
Expected Result	Error message: "Please enter a valid email address". User stays on login page.
Actual Result	(To be filled after execution)
Status	Pass / Fail

Field	Value
TC-ID	LOGIN_003
Title	Login with correct email but wrong password
Preconditions	User exists: testuser@example.com.
Test Steps	1. Enter testuser@example.com 2. Enter WrongPass 3. Click "Login".
Expected Result	Error message: "Invalid email or password". No account lockout (for first 3 attempts).
Actual Result	(To be filled after execution)
Status	Pass / Fail

Activity

- **Design a complete Test Design Specification** for the "Add to Cart" feature:
  - List **all the test conditions** you can identify from the business rules (aim for at least 5).
  - For **each condition**, write a full test case in table format, following the TC-ID / Title / Preconditions / Test Steps / Expected Result / Status structure.
  - Include **at least 2 positive test cases** and **at least 3 negative test cases** (e.g., exceeding stock, out-of-stock item, guest checkout flow).
  - Assign a unique, consistent TC-ID naming scheme (e.g., CART_001, CART_002, …).
- **Justify your design**: In 3–4 sentences, explain how your set of test cases achieves reasonable coverage of the feature's business rules, and identify any edge case you deliberately included or any you decided was out of scope (and why).